

Traffic Intersection Simulator

Internship Technical Assignment – Round 1

Traffic Intersection Simulator

Internship Technical Assignment – Round 1

1. Purpose

This assignment is intended for Round 1 candidates in our internship selection process. You have 4 weeks to explore, design, and prototype a miniature traffic-intersection simulator.

What we care about most:

- ✓ Your thought-process
- ✓ Your creativity
- ✓ Your code quality & cleanliness

2. Context

You'll simulate a standard four-way junction (North–South–East–West) with the following basic traffic light behavior:

Phase A (X_1 seconds):

North + South → ● green

East + West → ● red

Phase B (X_2 seconds):

North + South → ● red

East + West → ● green

Each direction has its own vehicle arrival rate ($A_1 \dots A_4$ cars/minute). A car takes S_1 seconds to pass the stop-line, departing in FIFO order.

You may extend the logic (amber lights, pedestrian crossings, adaptive timing, etc.) – just document your assumptions.

Optional V2 · Realistic Intersection Mode

(Challenge extension – attempt only if you have extra time)

Upgrade your design to represent a **true 4-way intersection** where each approach may have its **own signal group(s), phase order, and duration**. Consider adding:

- Separate queues for left/straight/right movements with lane specific lights.
- Conflict free scheduling (no overlapping greens that would cause collisions).
- Adaptive timing driven by virtual vehicle detectors.
- Distinct arrival/service rates per movement.
- Configurable parameters loaded at runtime from a file, CLI, or lightweight database.

3. Your Mission

Deliver a complete mini-project, from initial idea to working code:

3.1 Concept & Assumptions

- State the problem in your own words.
- Clearly list any extra rules, simplifications, or assumptions.

3.2 UML & Architecture

- Provide at least a Class Diagram of the core system
- A Sequence or Activity Diagram for one full light cycle

3.3 Prototype (Java or Kotlin)

- Prioritize clean, idiomatic code and tests
- Optionally use Concurrency (Java Threads, Kotlin Coroutines)
- Optionally include Persistence (in-memory or runtime DB)
- Project must be runnable in < 5 minutes, with Gradle or Maven

3.4 README / Dev Guide

- Explain build & run process
- Show how to configure timing & arrival rates
- Describe architecture, trade-offs, and future improvements

3.5 Bonus (Optional but Appreciated)

- Visualization (e.g., ASCII, simple GUI)
- Metrics logging/export
- CI setup (GitHub Actions, etc.)
- Dockerization

4. Evaluation Criteria

Weight | Area | What We Look For

-----|-----|-----

30 %	Design soundness	Cohesive model, abstraction, flexibility
25 %	Code quality	Clean style, idiomatic Java/Kotlin, testing
20 %	Creativity	Thoughtful extensions, smart edge cases
15 %	Documentation	Clear diagrams, good comments, helpful README
10 %	Professionalism	Git usage, commits, directory structure

We review the whole package – not just your code.

5. Submission Guidelines

- Push your solution to a **public or private Git repository** and share access.
- Include:
 - /design folder with UML files (.png/.svg + source)
 - Source code, unit tests, and build scripts
 - README.md with build/run/config instructions

6. After Submission

During the second interview round, we will:

- Discuss your approach & design decisions
- Ask about trade-offs and potential improvements
- Look at your project like a real-world team handoff

Final Notes

Good luck — and have fun turning red lights green! 🚦

Your creativity and craftsmanship matter more than rigid correctness.